

Generic Template — Format Guide & Test Harness

Every template file in this folder follows the same two-part structure:

- 1. **Template algorithm + template code** — the reusable pattern, with specific LeetCode problems used as concrete examples inside the code.
- 2. **Quick reference table + question/solution** — a table listing every problem (with a *Variation from Template* column), followed by each problem's full solution where deviations are marked `// ← VARIATION: explanation`.

Generic Test Harness

Use this boilerplate to run any single problem locally. Replace `data / solve()` with the problem's input and logic.

```
class Question {  
  
    private int data;  
  
    public Question(int data) {  
        this.data = data;  
    }  
  
    public void solve() {  
        // TODO  
    }  
}  
  
public class Solution {  
  
    public static void main(String[] args) {  
        Question q = new Question(0);  
        q.solve();  
    }  
}
```

Naming Conventions (enforced across all template files)

Context	Convention
Index variables	<code>i, j, k</code> (never <code>left, right, low, mid, high</code>)
Linked list	<code>sentinel, previous, current</code>
Queue	<code>Queue<Integer> queue = new ArrayDeque<>();</code>
BFS over tree	<code>Queue<TreeNode> queue = new ArrayDeque<>();</code>
Grid directions	<code>int[] dr = {1,-1,0,0}, dc = {0,0,1,-1};</code> (DOWN, UP, RIGHT, LEFT); iterate <code>for (int dir=0; dir<4; dir++)</code> using <code>i+dr[dir], j+dc[dir]</code>
Character frequency	<code>count[ch - 'a']++;</code>
Digit frequency	<code>count[ch - '0']++;</code>
Char to integer build	<code>num = num * 10 + (ch - '0');</code>

Common Java API Cheat-Sheet

The APIs used over and over across these templates. Grouped by what you reach for.

Initialize the core structures

```
Map<Integer, Integer> map = new HashMap<>(); // key → value
Map<Integer, List<Integer>> adj = new HashMap<>(); // adjacency / buckets
Set<Integer> set = new HashSet<>(); // membership
List<Integer> list = new ArrayList<>(); // dynamic array

Deque<Integer> stack = new ArrayDeque<>(); // STACK: push / pop / peek
Deque<Integer> queue = new ArrayDeque<>(); // QUEUE: offer / poll / peek
// (ArrayDeque beats java.util.Stack / LinkedList)

PriorityQueue<Integer> minHeap = new PriorityQueue<>(); // min-heap (default)
PriorityQueue<Integer> maxHeap = new PriorityQueue<>(Collections.reverseOrder()); // max-heap
PriorityQueue<int[]> pq = new PriorityQueue<>((a, b) -> a[1] - b[1]); // by field, min-first

TreeMap<Integer, Integer> tmap = new TreeMap<>(); // sorted keys: floor/ceiling/first/last
TreeSet<Integer> tset = new TreeSet<>(); // sorted set: floor/ceiling/higher/lower
LinkedHashSet<Integer> lset = new LinkedHashSet<>(); // insertion-ordered set (LFU buckets)
Random rand = new Random(); // rand.nextInt(n) → 0..n-1 (RandomizedSet)

int[] arr = new int[n]; // defaults to 0
boolean[] seen = new boolean[n]; // defaults to false
int[][] grid = new int[m][n]; // 2D, all 0
int[] dr = {1,-1,0,0}, dc = {0,0,1,-1}; // DOWN,UP,RIGHT,LEFT; iterate for(dir=0..3) i+dr[dir],
```

Map / Set / List operations

```
// — Map —
map.put(k, v);
map.getOrDefault(k, 0); // safe read with default
map.merge(k, 1, Integer::sum); // ← counting frequency (most common)
map.computeIfAbsent(k, x -> new ArrayList<>()).add(v); // ← build adjacency / buckets
map.putIfAbsent(k, v); // ← write only first occurrence (keep earliest in
map.containsKey(k); map.remove(k);
for (Map.Entry<Integer,Integer> e : map.entrySet()) { e.getKey(); e.getValue(); e.setValue(v); }
map.keySet(); map.values();

// — Set —
set.add(x); set.contains(x); set.remove(x);

// — List —
list.add(x); list.get(i); list.set(i, x);
list.remove(list.size() - 1); // ← pop in backtracking
list.size(); list.isEmpty(); list.addAll(other);
Collections.sort(list); // ascending
list.sort((a, b) -> b - a); // custom / descending
Collections.reverse(list); // ← reverse path after building it backwards
```

Stack / Queue / Heap (all via the same two ops)

```
// Deque as STACK (LIFO)                // Deque as QUEUE (FIFO)
stack.push(x);                          queue.offer(x);
int top = stack.pop();                  int head = queue.poll();
int look = stack.peek();               int look = queue.peek();
stack.isEmpty();                       queue.isEmpty();

// Deque BOTH ENDS – needed for the MONOTONE DEQUE (sliding-window max/min)
// and for addFirst-style level building (zigzag BFS).
deque.offerLast(x); deque.pollLast(); deque.peekLast(); // back (push/pop/peek tail)
deque.offerFirst(x); deque.pollFirst(); deque.peekFirst(); // front (push/pop/peek head)
deque.addLast(x); deque.addFirst(x); // same as offer*, throw if capacity-bou

// PriorityQueue (heap)
pq.offer(x); int best = pq.poll(); int look = pq.peek(); pq.size();
pq.remove(x); // ← O(n) removal of an arbitrary element (lazy-deletion alternative in #480)
```

Integer / Long limits & overflow-safe idioms

```
Integer.MAX_VALUE; // 2147483647 (2^31 - 1) – sentinel for "min so far"
Integer.MIN_VALUE; // -2147483648 (-2^31) – sentinel for "max so far"
Long.MAX_VALUE; // when sums can overflow int, accumulate in long
int k = i + (j - i) / 2; // ← avoid (i + j) overflow in binary search
Integer.compare(a, b); // ← overflow-safe comparator; use instead of (a - b)
(a, b) → Integer.compare(a[1], b[1]); // ← safe PriorityQueue/Arrays.sort comparator
Double.compare(a, b); // ← comparator for double[] heaps (probability, median)
```

Char ↔ int ↔ String conversions

```
ch = 'a'; // 'a'..'z' → 0..25 (lowercase bucket index)
ch = '0'; // '0'..'9' → 0..9 (digit value)
(char) ('a' + i); // 0..25 → 'a'..'z'
num = num * 10 + (ch - '0'); // ← build a number digit by digit

Integer.parseInt("123"); // String → int (throws on bad input)
Integer.valueOf(123); // int → Integer (boxed)
String.valueOf(123); // int/char/etc → String
Character.getNumericValue('7'); // char → 7

// char classification
Character.isDigit(ch); Character.isLetter(ch); Character.isLetterOrDigit(ch);
Character.isWhitespace(ch); Character.toLowerCase(ch); Character.toUpperCase(ch);
```

String & StringBuilder

```
s.length(); s.charAt(i); s.substring(i, j); // [i, j)
s.toCharArray(); s.split(" "); s.equals(t); s.compareTo(t);
s.indexOf("ab"); s.isEmpty();
s.startsWith("ab"); s.endsWith("z"); s.contains("x"); // prefix / suffix / substring tests
s.toLowerCase(); s.toUpperCase(); // case-normalize (e.g. case-insensitive compare)
s.trim(); s.replace('a', 'b'); s.repeat(n); // strip ends / swap chars / repeat

StringBuilder builder = new StringBuilder();
builder.append(x); builder.reverse(); builder.toString();
builder.charAt(i); builder.setCharAt(i, c); builder.deleteCharAt(i); builder.length();
String.join(",", list); // list → "a,b,c"
```

Arrays utilities

```
Arrays.sort(arr); // ascending in place
Arrays.sort(intervals, (a, b) -> a[0] - b[0]); // 2D by first field
Arrays.fill(dp, Integer.MAX_VALUE); // seed a dp/dist array
Arrays.equals(window, need); // ← element-wise array compare (anagram check)
Arrays.copyOf(arr, len); Arrays.copyOfRange(arr, i, j);
System.arraycopy(src, srcPos, dst, dstPos, len); // ← fast block copy (merge step)
Arrays.asList(1, 2, 3); // fixed-size List view
int sum = Arrays.stream(arr).sum();
int max = Arrays.stream(arr).max().getAsInt();

// List<Integer> ↔ int[]
int[] a = list.stream().mapToInt(Integer::intValue).toArray();
List<Integer> l = Arrays.stream(a).boxed().collect(Collectors.toList());
```

Math & bit operations

```
Math.max(a, b); Math.min(a, b); Math.abs(x);
Math.pow(a, b); Math.sqrt(x); Math.ceil(x); Math.floor(x);

n & (n - 1); // clear lowest set bit
n & (-n); // isolate lowest set bit
1 << i; // bit i set (use 1L << i for i ≥ 31)
(n >> i) & 1; // read bit i
(n & 1) == 0; // ← even (last bit 0)
(n & 1) == 1; // ← odd (last bit 1)
n >> 1; // ← divide by 2 (drop last bit); n << 1 = multiply by 2
Integer.bitCount(n); // # of set bits
Integer.toBinaryString(n);
```

Sorted-structure navigation (TreeMap / TreeSet)

```
tmap.firstKey(); tmap.lastKey();
tmap.floorKey(x); // largest key ≤ x tmap.ceilingKey(x); // smallest key ≥ x
tmap.lowerKey(x); // largest key < x tmap.higherKey(x); // smallest key > x
tset.floor(x); tset.ceiling(x); tset.lower(x); tset.higher(x); tset.first(); tset.last();
```

Optimization idioms

Common speed/space wins used throughout the templates.

```
// — NEGATIVE-SAFE MODULO — replaces the manual ((x % n) + n) % n
Math.floorMod(x, n); // always in [0, n-1] even when x < 0 (prefix-sum % k)

// — int[] INSTEAD OF HashMap for a bounded key space —
int[] count = new int[26]; // 0(1) no-boxing freq vs HashMap<Character,Integer>
count[ch - 'a']++; // also faster to compare: Arrays.equals(a, b)

// — 1D ROLLING ARRAY for DP — drop a dimension when row i only needs row i-1
int[] dp = new int[n + 1]; // 0(n) space instead of 0(m·n)
for (...) for (int j = target; j >= w; j--) dp[j] += dp[j - w]; // 0/1 knapsack: iterate j DOWN

// — FIXED-SIZE HEAP for top-k — 0(n log k) beats sorting 0(n log n)
if (heap.size() > k) heap.poll(); // keep only k best; min-heap for "k largest"

// — LAZY DELETION over pq.remove(obj) — pq.remove is 0(n); skip stale entries instead
if (entry[0] != dist[node]) continue; // Dijkstra: ignore outdated heap entries

// — ArrayDeque over Stack / LinkedList — faster, no synchronization, both ends 0(1)
Deque<Integer> stack = new ArrayDeque<>(); // never use java.util.Stack

// — StringBuilder over String += in a loop — 0(n) vs 0(n²)
StringBuilder builder = new StringBuilder(); // append, then builder.toString() once

// — EARLY EXIT / PRUNING — return the moment the answer is decided
if (found) return result; // e.g. backtracking bounds
```

Template Index

File	Category	Core Pattern
binary_search.md	Binary Search	Closed <code>[i, j]</code> , half-open, minimize/maximize answer space
two_pointers.md	Two Pointers	Opposite ends, same-direction write pointer, Dutch flag
prefix_sum_hashmap.md	Prefix Sum	Count/length with HashMap, tree path sums
sorting.md	Sorting	Merge sort, quick sort, quick select, count-during-merge
sliding_window.md	Sliding Window	Fixed, max-variable, min-variable windows
linked_list.md	Linked List	Delete/filter, reversal, fast/slow, merge, composite
dfs.md	DFS / Backtracking	Tree return-up/pass-down, grid DFS, graph 3-color, backtracking, BST
bfs.md	BFS	Level-order with size snapshot, position indexing
graph.md	Graph	BFS shortest path, topo sort, union find, Dijkstra, bipartite, Prim's MST
dynamic_programming.md	Dynamic Programming	1D/2D/grid/interval DP, knapsack, state machine
stack_queue.md	Stack / Queue / Heap	Monotone stack/deque, two heaps, priority queue
greedy.md	Greedy	Sort-by-end, sort-by-start, sort + heap, non-interval
string.md	String	Frequency counting, hashing, bucket sort, expand-from-center
bit_manipulation.md	Bit Manipulation	XOR cancel, mod-k state machine, bitmask as set, power checks
design.md	Design	HashMap + linked list (LRU), frequency maps (LFU), array + map
math.md	Math	Fast power, base conversion, sieve, digit manipulation